Failure-by-Language: Reproducible Evaluation of Python-like Confusion Language in LLM Code Generation

Jaeyeong CHOI
DGIST EECS

jaeyeong2022@dgist.ac.kr

***Abstract*—Recent studies on AI-assisted coding suggest a persistent gap between short-term productivity gains and durable programming comprehension. This paper proposes a reproducible benchmark for syntax-constrained code generation: language models are instructed to write Python-like programs under transformed-token rules. We present an OCaml/CLI pipeline with lineage-aware task metadata, schema validation, and metric snapshot automation. Current empirical results are limited by temporary LLM API authentication failures; however, fixture replay experiments still provide a controlled baseline for measuring alias compliance, Python reversion, and execution success behavior.**

***Index Terms*—confusion language, LLM code generation, syntax perturbation, benchmark, reproducibility, program synthesis**

I. Introduction

Recent studies on AI-assisted coding indicate a persistent tradeoff: immediate coding productivity may rise while durable comprehension and independent debugging skills lag, especially for beginners. This motivates evaluation under constrained syntax settings rather than only canonical benchmarks.

This study asks a strict question: when a model is instructed to produce code in a Python-like language with transformed syntax, does it follow the transformed language rules, or revert to canonical Python conventions?

We define a three-layer benchmark stack:

1) **Specification layer**: one-to-one token aliases and collision constraints.
2) **Execution layer**: OCaml-based transform, validate, and roundtrip tooling.
3) **Measurement layer**: summary validation, schema checks, and metric snapshots.

Our goal is not absolute model ranking. We isolate where constrained-language compliance fails and how failures relate to execution outcomes.

II. Related Work

A. Code-generation benchmarks and robustness

Code-generation benchmarks and model baselines (e.g., HumanEval and MBPP) motivate short-horizon program synthesis evaluation. [1], [2]

B. Code representation models under perturbation

Pretraining work such as *CodeBERT*, *CodeT5*, and *UniXcoder* improves code representation transfer, while preserving language semantics under transformed-token constraints remains underexplored in these settings. [3], [4], [5]

C. Instruction and alignment effects

Instruction-following tuning improves broad task compliance, yet its effect on non-standard syntax transformations has not been studied sufficiently. [6], [7]

D. Repair-oriented generation

Editing and infilling methods (e.g., InCoder) indicate that bidirectional context can support local correction, relevant to our roundtrip diagnosis design. [8]

E. Software-engineering realism

SWE-bench demonstrates the gap between benchmark-scale success and real-world patching complexity, reinforcing our emphasis on strict artifact and lineage control. [9]

F. Constraint-aware decoding

Recent progress in constrained decoding suggests adding semantic constraints can complement syntax-layer methods, an important extension direction for confusion-language pipelines. [10]

III. Method

A. Research Question and Hypotheses

We study whether code LLMs obey transformed Python-like syntax under explicit alias constraints.

RQ1: Under strict alias instructions, how often do models retain transformed tokens vs. revert to canonical Python keywords?

RQ2: Are failures dominated by infrastructure issues or by behavioral non-compliance after infrastructure stabilization?

H1: Even with valid API execution, strict alias compliance remains low because models revert to dominant Python priors.

B. Language Setup and Constraints

We define a confusion language by replacing selected Python tokens with aliases. Constraints:

1) one-to-one mapping between each source token and alias;
2) no alias sharing among control keywords (e.g., `if`/`elif`);
3) token-boundary replacement;
4) comment/string literal protection.

C. Pipeline Architecture

The evaluation stack combines OCaml transformation and Python orchestration:

- **Transform/validate layer**: alias checks, collision prevention, roundtrip normalization;
- **Batch execution layer**: prompt-version evaluation and result serialization;
- **Schema gate layer**: summary payload and metric schema validation.

Key scripts include `run_gpt54_eval.py`, `batch_report_summary.py`, `generate_metric_snapshot.py`, `validate_summary_payload.py`, and `validate_metric_schema.py`.

D. Metrics and Judge Protocol

We use two metric families:

- **Fixture aggregate metrics**: ACR, PRR, ESR, MFB, LGP.
- **Prompt-batch judge outcomes**: pass/fail with violation taxonomy.

Judge protocol checks:

1) strict alias presence and original keyword leakage;
2) alias-to-python normalization validity (AST parseability);
3) target skeleton consistency for the benchmark task pattern.

E. Threats to Validity

Internal validity: false negatives may occur if judge rules are stricter than practical human acceptance.

Construct validity: pass/fail under one task skeleton does not represent full software engineering competence.

External validity: current runs focus on a constrained prompt-family and may not generalize across broader task distributions.

IV. Experiments

A. Experiment Phases

The project proceeded in phase order:

- **Phase A (stability)**: CI and schema/lineage fixes;
- **Phase B (fixture baseline)**: replay-based metric snapshots;
- **Phase C (real-call scaling)**: full-range prompt batches across multiple model settings.

TABLE I
Fixture replay baseline metrics.

Metric	Value	Note
ACR	0.333	fixture replay (`fixture-task-set-v1`)
PRR	0.667	same run condition
ESR	0.667	same run condition
MFB	65.0	manual correction burden estimate
LGP	0.667	learning proxy metric from replay context

B. Input Set and Coverage

Prompt inputs are versioned as `v1`–`v120`. We execute both segmented and full-range runs, then archive model-specific JSON/CSV/Markdown artifacts.

C. Execution Conditions

- Models tested: `gpt-4o-mini`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`.
- Prompt condition: strict compliance with transformed token mapping.
- Output policy: code-only response, deterministic judge pipeline.

D. Research Process Notes

Early runs were blocked by API authentication mismatch (HTTP 401). After key-path correction and environment sync, real-call execution became stable and shifted bottlenecks to model behavior under strict constraints.

E. Artifact and Audit Policy

Every run stores:

1) raw JSON result payload,
2) tabular summaries (CSV/MD),
3) aggregate status snapshots for longitudinal tracking.

This enables post-hoc audit of each claim in the paper against committed artifacts.

V. Results

A. Real-call full-range batches (v1–v120)

After key-path recovery, we executed full-range real-call batches for four model settings:

- `gpt-4o-mini`: total=120, passed=0, failed=120
- `gpt-4.1-mini`: total=120, passed=0, failed=120
- `gpt-4.1-nano`: total=120, passed=0, failed=120
- `gpt-5.4-mini`: total=120, passed=0, failed=120

Total accumulated real-call samples: **480**. In these full-range runs, HTTP authentication failures were removed; dominant failures moved to judge-side violations.

B. Observed violation profile

Representative high-frequency violation categories include:

- `original keyword used`
- `python parse failed after normalization`
- alias-missing and roundtrip mismatch derivatives

For parse-failure counts, `gpt-5.4-mini` (95) and `gpt-4.1-mini` (96) were lower than `gpt-4o-mini` (109) and `gpt-4.1-nano` (111), but all remained at 0 pass under the current strict judge protocol.

C. Interpretation

The bottleneck has shifted from infrastructure authentication to strict compliance under transformed syntax constraints. This confirms that the current challenge is primarily behavioral (rule adherence), not endpoint availability.

VI. Discussion

The current stage contributes a robust benchmark scaffold *and* a meaningful real-call accumulation (480 samples), but still no passing cases under strict compliance.

A. What is now validated

- Replay-side reproducibility is stable (schema checks, lineage, fixture snapshots).
- Real-call execution is operational after key-path recovery (HTTP auth failures removed in full-range runs).
- Failure concentration is now clearly behavioral: keyword reversion and normalization-parse breakdown.

B. What remains limited

- All four tested model settings currently show 0 pass under strict judge criteria.
- Cross-condition prompt design is still narrow (few-shot/repair-loop variants not yet fully explored).

C. Methodological implication

For constrained-language evaluation, provenance and schema controls are necessary but not sufficient. Once infra is stable, model behavior can still collapse under strict rule adherence. Therefore, next experiments should explicitly separate prompt-design effects, decoding constraints, and judge strictness sensitivity.

VII. Conclusion

This project delivers a reproducible confusion-language benchmark stack with collision-safe alias policies, provenance-rich output, and schema-enforced measurements.

Current quantitative evidence is limited: fixture replay indicates non-trivial non-compliance and Python reversion under transformed syntax, while real API-based model expansion awaits valid credentials.

Next steps are to (1) complete cross-condition batch comparisons, (2) report mismatch decomposition over larger prompt sets, and (3) integrate learner-centered diagnostics (manual-fix burden and learning-gain trajectory) into the same pipeline.

VIII. Research Process Traceability

We keep a direct trace between experimental decisions and artifacts. Key sources include the live status dashboard, daily research logs, and model-specific prompt-batch outputs. This paper only claims results that can be mapped to committed files in `docs/research/results/` and verified by schema checks.

IX. Reproducibility and Artifact Availability

All scripts, logs, and metrics are versioned in the repository. Representative paths:

- Tooling: `ocaml-confusion-lang/`
- Results: `docs/research/results/`
- Logs: `docs/research/log/`
- Status snapshot: docs/research/context-state.json
- Paper draft: docs/research/papers/v2.md

References

[1] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] J. Austin *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[3] Z. Feng *et al.*, "Codebert: A pre-trained model for programming and natural languages," in *EMNLP-Findings*, 2020.

[4] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *EMNLP*, 2021.

[5] D. Guo *et al.*, "Unified cross-modal pre-training for code representation," 2022.

[6] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," 2022.

[7] T. Brown *et al.*, "Language models are few-shot learners," arXiv preprint arXiv:2005.14165, 2020, transformer scaling and in-context learning baseline for subsequent instruction-following work.

[8] D. Fried *et al.*, "Incoder: A generative model for code infilling and synthesis," *arXiv preprint arXiv:2204.05999*, 2022.

[9] C. Jimenez *et al.*, "Can language models resolve real-world github issues?" *arXiv preprint arXiv:2310.06770*, 2023.

[10] J. Muendler *et al.*, "Type-constrained code generation with language models," 2025, type-aware decoding constraints for code generation (emerging work).